

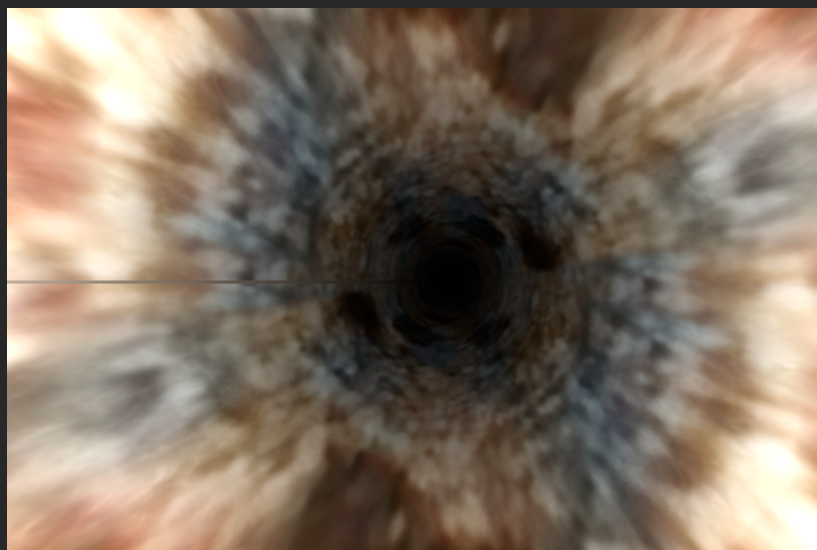
Inigo Quilez :: articles :: tunnel artifact - 2013



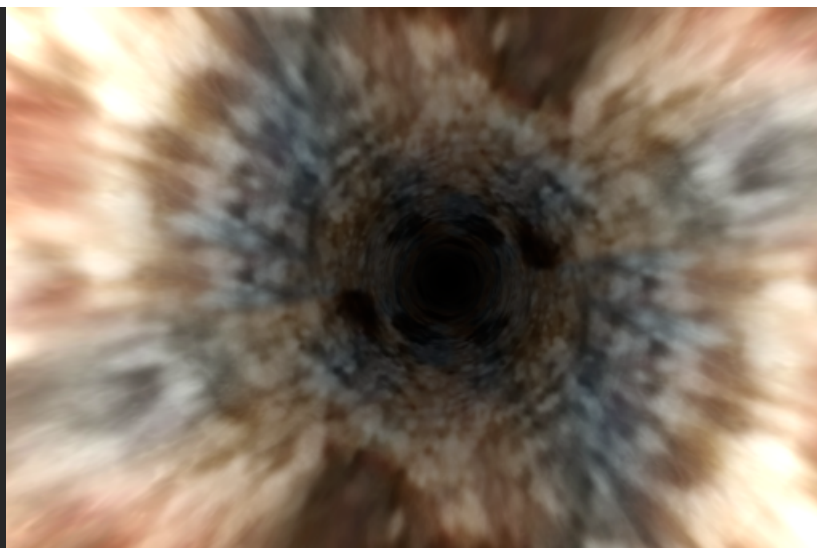
Intro

Texturing a polygonal cylinder might be more difficult than it seems, due to the seam which is really obvious under mipmapping. One way to fix it is to duplicate some vertices and have unique UVs in them, but that can be expensive or inconvenient (for example, it can mess up the way the normals of the mesh are computed).

A very similar situation happens when rendering a raytraced cylinder, such as when creating a tunnel effect, such as those we usually see in small demos and intros in the demoscene. One easy way to achieve the effect is by simply performing a **plane deformation** that converts the cartesian coordinates to polar and then inverts the radius. This is equivalent to performing a raytrace from the origin into a cylinder and then apply cylindrical mapping to it. When done in the GPU one probably wants to enable mipmapping in the texture of the tunnel in order to get smooth antialiased pixels in the back of the tunnel, far from the camera. However, when doing so, one often gets an artifact in one side of the tunnel where a whole line of pixels (2 lines, to be exact) displays incorrect colors.



Rendering with line artifact

*Corrected rendering*

The Problem

The formula for the tunnel usually is something like this:

```
vec3 doTunnel( sampler2D sam, in vec2 uv, in float time )
{
    // get polar coordinates
    float a = atan( uv.y, uv.x );
    float r = length( uv );

    // pack and animate
    uv = vec2( 1.0/r + time, a/3.1415927 );

    // fetch from texture
    return texture2( sam, uv ).xyz;
}
```

The code makes sense, but it's buggy as we know from the rendered images. The artifact happens only in some conditions. Say the center of the tunnel falls between two pixels p , and $p+1$. If p is an odd number, then the artifact won't manifest. However, if p is an even number, the line artifact appears.

The reason is that the GPU computes uv derivatives only in blocks of 2×2 pixels. As we know, uv derivatives are needed to estimate how big the footprint of a pixel is in texture space, or in other words, to know how many texels in the texture will be needed during filtering to render this pixel. This information is only computed every other pixel, due to the way the hardware threads work together in the GPU. But basically, if the values of the uv change more than they should for any reason within one of those 2×2 pixels, we'll have problems.

Of course, that's exactly what can happen when the `atan()` changes from π to $-\pi$, right at the branch (the $-x$ axis). While everywhere else `atan()` produces smoothly change values (as you move across pixels `atan()` produces slowly changing values), for the pixels right above and below the $-x$ axis things change abruptly.

Now, as said before, we can get lucky and it might happen that this change happens exactly between two groups of 2×2 pixels, and therefore never gets registered. In that case, the artifact doesn't happen. But if we are unlucky and we placed our tunnel in the screen such that the $-x$ axis falls exactly inside a 2×2 pixel group, or in other words, if it happens between pixels p and $p+1$ with an p even number, then the 2×2 pixel quad will register a difference of 2π in one of the coordinates. That will be enough to push the texture lookup to the smallest levels of the LOD/mip chain, which usually contains the average color of the texture, resulting in the artifact.

The Fix

The fix, as with **all other artifacts related to mipmapping and discontinuities in the texture coordinates**, is to compute these by hand.

In our case, since the tunnel is symmetric, we can use the right side of the tunnel for the derivative computation, and then propagate it to the left side of the tunnel:

```
vec3 doTunnel( sampler2D sam, in vec2 uv, in float time )
{
    // get polar coordinates
    float a = atan( uv.y, uv.x );
    float b = atan( uv.y, abs(uv.x) );
    float r = length( uv );

    // pack and animate
    vec2 uvL = vec2( 1.0/r + time, a/3.1415927 );
    vec2 uvR = vec2( 1.0/r + time, b/3.1415927 );

    // fetch from texture
    return textureGrad2( sam, uvL, dFdx(uvR), dFdy(uvR)).xyz;
}
```

This fixes the artifact. Of course, if you were concerned by the double call to `atan()` (you probably shouldn't), you can probably compute only one the right side of it and then adjust for the left side based on some basic

trigonometry.

This is a live running example of this simple trick, running in Shadertoy:

<https://www.shadertoy.com/view/Ms2SWW#>:



inigo quilez - learning computer graphics since 1994